Grounded Morphological Memory for Ge'ez→Amharic Translation

Plain-language overview of the work so far · Esubalew Chekol, MSc in AI, Addis Ababa University · 15 October 2026

Purpose of this document. A reader with no knowledge of the project history should be able to answer: what problem is studied, what each system is, how the systems differ, what was run, what was found, and what comes next. All numbers are on the development set; the test set has not been used.

1. The problem in one paragraph

Ge'ez is a morphologically rich Semitic language. One **root** (usually three consonants) produces many word forms. For example the root s-g-d ("bow, worship") gives ሰገደ (he bowed), ሰገዱ (they bowed) and ትሰግዱ (you bow). A translation system trained on 14,000 sentence pairs sees most of these forms rarely or never. The thesis asks: **if we give the translator an explicit memory that is organised by roots, does it recognise roots of unseen word forms, does it actually use that memory, and does translation get better?**

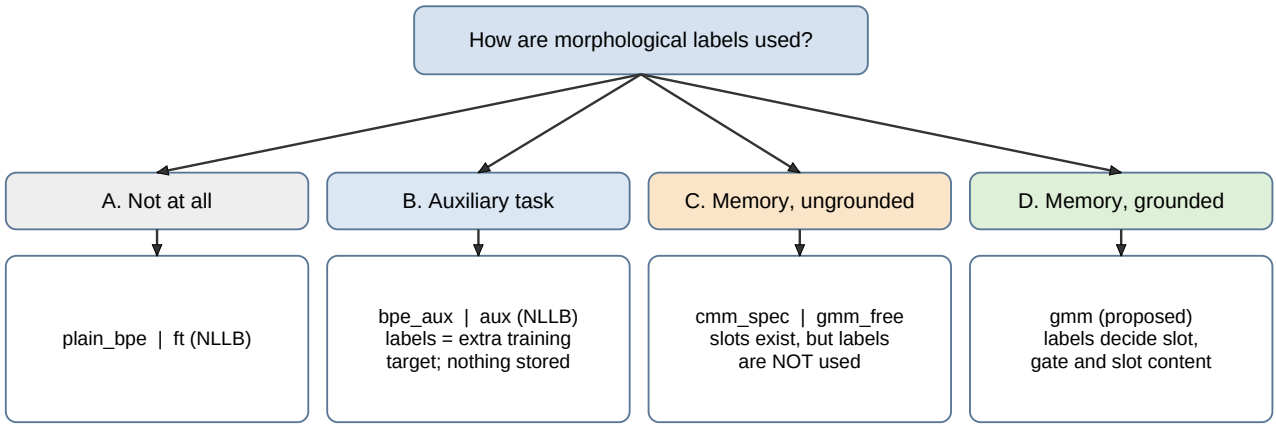
Short answer so far. Recognition: yes, clearly. Use: yes in the small model, hardly in the large pretrained NLLB model. Translation: a gain of about +0.4 chrF in the small model with 1,024 root slots (not yet significant across seeds); no gain in NLLB.

2. Terms used in this document

Term	Meaning in plain words
Root	The consonant skeleton shared by related words (s-g-d). Carries the core meaning.
Pattern	The template of vowels/affixes that turns a root into a specific form (tense, person, noun type). 16 pattern classes are used.
Morphological analyser	A separate model (the author's ByT5 analyser) that reads a word and predicts its root and pattern.
Label	The analyser's answer for a word, e.g. "root = s-g-d, pattern = perfective 3rd plural". Labels are used only during training.
Rootless word	Function words (prepositions, particles, pronouns), names and punctuation. They get label 0 = "no root".
Label inventory	Only the most frequent roots get their own label: top 255 (256-slot setting) or top 1,023 (1,024-slot setting). Rarer roots get 0.
Auxiliary task ("aux")	An extra small output layer trained to predict the label from the encoder. It only adds a training signal; nothing is stored or looked up during translation.
Memory	An extra block inside the encoder holding a bank of learned vectors (**slots**). For each word, the block picks a slot and adds its vector to the word's representation.
Router	The part of the memory that decides *which* slot a word uses.
Gate	A number between 0 and 1 that decides *how much* of the slot is added. Closed (near 0) means the memory is ignored for that word.
Ungrounded memory	Memory whose slots are learned freely; nobody tells it what a slot should mean.
Grounded memory	Memory where slot k is **tied to root k**: labels train the router (go to your root's slot), the gate (open only if the word has a root) and the slot content.
Detection (own-slot accuracy)	For words never seen in training: the share that the router sends to the slot of their true root. 0% = no root knowledge.
chrF / BLEU	Automatic translation-quality scores (higher is better). chrF counts matching characters, which suits rich morphology; it is the main score.
Seed	The random starting point of training. Re-running with another seed shows how much a result is due to luck (about ±0.23 chrF here).
Beam 5, no repeated 4-grams	The decoding method: keep the 5 best partial translations and forbid repeating any 4-word sequence. Used for all reported scores.

3. The four families of systems (taxonomy)

Every system is the **same translator** trained on the **same data**, with the same number of parameters (within 0.05%). They differ only in **how the morphological labels are used**:



Each family exists in the small model (20.3M) and, where marked, in NLLB-600M. Grounded memory comes with 256 or 1,024 root slots.

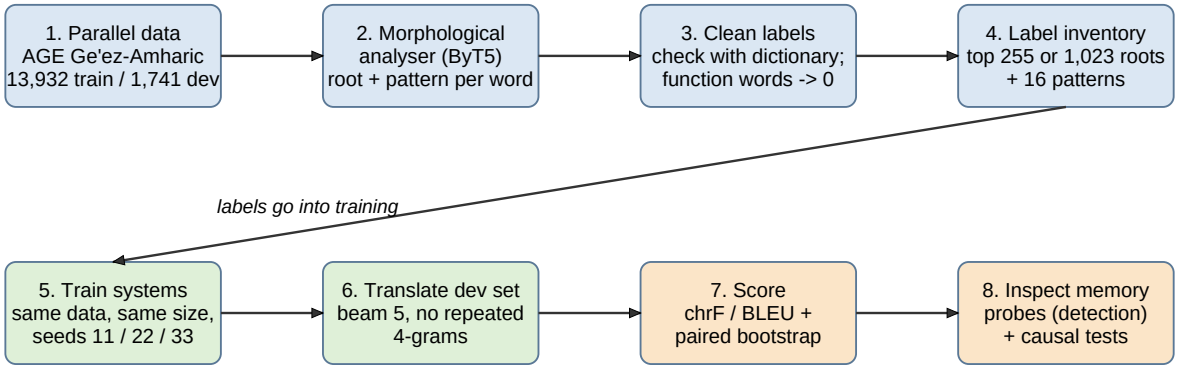
Why both B and C are needed as comparisons. If grounded memory (D) beats the auxiliary task (B), the gain is not just "labels help". If D beats ungrounded memory (C), the gain is not just "an extra memory block helps". Only a win over both shows that *tying slots to roots* matters.

Code name	Family	Model	What it is
plain_bpe	A	small	Transformer on subword tokens, no labels
bpe_aux	B	small	plain_bpe + auxiliary heads predicting root and pattern (strongest label baseline)
cmm_spec	C	small	memory module from the thesis specification, learned without labels
gmm	D	small and NLLB	grounded morphological memory (the proposal); the small version also keeps the auxiliary heads
ft	A	NLLB	NLLB-200-distilled-600M fine-tuned on AGE, no memory
aux	B	NLLB	ft + auxiliary heads
gmm_free	C	NLLB	exactly the gmm block and size, but no grounding
*_wide	-	both	the same system with 1,024 root slots instead of 256

3.1 How a word gets its label ("is this word grounded?")

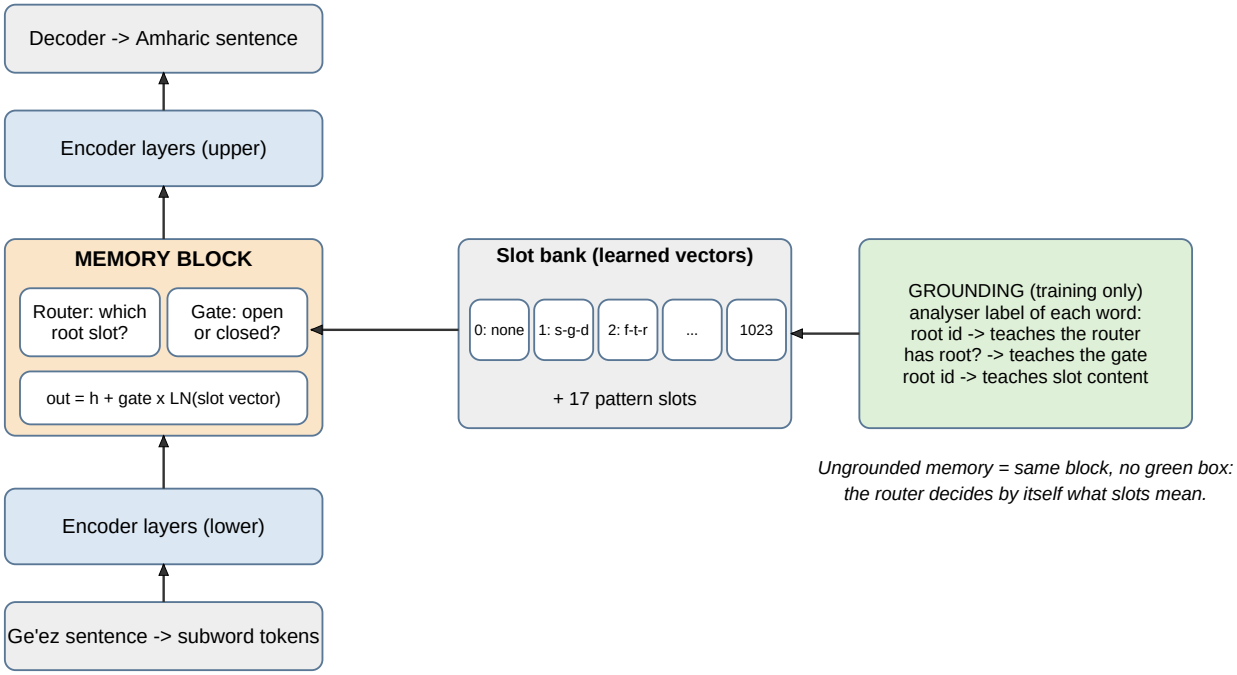
1) The analyser predicts a root and pattern for every training word. 2) The prediction is checked against the author's Ge'ez dataset (geez-dataset): dictionary forms take the curated root; a predicted root is kept only if its consonants actually occur in the word in order. 3) Function words, names, punctuation and non-Ge'ez tokens are forced to label 0. (Earlier labels wrongly gave roots to words like ውስተ "in", ከመ "like", እግዚአብሔር "God" and even the punctuation mark ።.) 4) The 255 or 1,023 most frequent roots get their own slot; all others share label 0. So a word is "grounded" when it has a verified root inside the inventory; for such words the gate should open, for all others it should stay closed.

4. Workflow



Steps 1–4 build the labels once. Step 5 trains every system on identical data. Steps 6–7 measure translation; step 8 opens the memory to check what it learned and whether the translator depends on it.

5. Architecture of the grounded memory



Ungrounded memory = same block, no green box: the router decides by itself what slots mean.

The memory block sits between encoder layers. For each token, the **router** chooses a root slot (and a pattern slot), the **gate** decides how much to add, and the result is added back to the token representation. Grounding (green) is used only while training: at translation time the model has no access to labels and must recognise the root by itself, which is why detection on *unseen* words is the key test. In NLLB the block is inserted at encoder layers 5 and 11 and starts switched off (zero/low initialisation) so it cannot harm the pretrained model at the start.

6. What was run, in order

Stage	What	Outcome
G0 runs (12–13 Oct)	All systems, small model (seeds 11, 22, 3 seeds of plain_bpe) and NLLB (ft, aux, gmm, gmm_free)	Labels help (+1.3 to +1.9 chrF over plain_bpe). gmm detected roots well but translated 0.4 chrF *worse* than bpe_aux.
Evaluation audit	Seed repeats; repetition analysis	Greedy outputs looped in 49–58% of sentences, so all scores were redone with beam 5 + no repeated 4-grams. Single-seed differences below about 0.8 chrF are noise.
Bug review (14 Oct)	Inspection of the G0 memory code	Only 44–52% of tokens got a training signal; slot 0 was never trained; sharpening applied to all words; labels gave roots to function words.
G1 fixes	Clean labels, "no root" = 0, supervised gate, slot-content loss, normalisation change	All introduced together (individual effects not yet separated).
G1 runs (14–15 Oct)	Small: bpe_aux, cmm_spec, gmm at 256 slots (seeds 11, 22); bpe_aux, gmm at 1,024 slots (seeds 11, 22, 33). NLLB: gmm_free, gmm, gmm_wide (seeds 11, 22)	Results in section 7. G0 numbers are kept only as history.

7. Results (current code, G1)

7.1 Does the memory detect roots? (share of unseen words routed to their own root slot; gate = average opening)

Setting	Grounded gmm: detection	Gate on rooted / rootless words	Ungrounded control
Small, 256 slots	78–80% (all-word consistency 0.90)	0.60–0.63 / 0.08	cmm_spec: consistency 0.24 (chance 0.18–0.21); gate 0.13 / 0.15
Small, 1,024 slots	65–68% (consistency 0.82–0.84)	0.60–0.66 / 0.08–0.09	-
NLLB, 256 slots	88%	0.87 / 0.07	gmm_free: 0.2%; gate does not separate
NLLB, 1,024 slots	80%	0.89–0.90 / 0.10	-

7.2 Does the translator use the memory? (tests on trained models, no retraining)

Test	Small gmm	NLLB gmm
Scramble the slot of root words: extra loss on root tokens / other tokens	+1.06 to +1.14 / about 0 (nats)	-
Switch memory off for the word: correct word lost	35–41%	1–3%
Put another root's slot in: correct word lost	52–56%	1–3%

The small translator clearly depends on the memory; NLLB already knows these words and largely ignores it.

7.3 Does translation improve? (chrF; mean over seeds; 95% interval from resampling dev sentences)

Setting	gmm (D)	bpe_aux (B)	cmm_spec / gmm_free (C)	D minus B
Small, 256 slots (2 seeds)	14.88	14.94	14.91	–0.06 [–0.30, +0.18]: tie
Small, 1,024 slots (3 seeds)	15.39	14.97	-	**+0.41 [+0.23, +0.60]**; per seed –0.02, +0.60, +0.67
NLLB, 256 slots (2 seeds)	32.29	aux –0.5 to –2.6 vs ft (G0)	gmm_free 32.48	D minus C: –0.19 [–0.41, +0.05]
NLLB, 1,024 slots (2 seeds)	32.50	-	gmm_free 32.48	D minus C: +0.02 [–0.19, +0.25]

Plain fine-tuned NLLB (ft) scores 32.33 on average; no memory system differs from it significantly. Zero-shot NLLB scores 13.5.

8. Status of each claim

Claim	Status	Main evidence
Grounded slots recognise roots, also of unseen words	Supported	65–88% detection vs about 0% ungrounded
The gate learns which words have roots	Supported	0.60–0.90 vs 0.07–0.10
The small translator uses the memory	Supported	removing/scrambling slots breaks 35–56% of root words
NLLB uses the memory	Not supported	97–99% of words survive without it
Grounding improves small-model translation	Partly supported	+0.41 chrF at 1,024 slots; 2 of 3 seeds; tie at 256
Grounding improves NLLB translation	Not supported	gmm ≈ gmm_free ≈ ft

9. Next plan (advisor meeting, 15 Oct)

9.1 Does better detection give better translation? (detection ablation)

The existing runs already have very different detection levels, but they also differ in model, label inventory and code, so they cannot answer this question:

Existing run	Detection	chrF
small cmm_spec (ungrounded)	~0%	14.91
small gmm, 1,024 slots	65–68%	15.39
small gmm, 256 slots	78–80%	14.88
NLLB gmm_free (ungrounded)	0.2%	32.48
NLLB gmm, 1,024 / 256 slots	80% / 88%	32.50 / 32.29

The planned experiment holds everything fixed and changes **only** detection, at the levels the advisor named (0%, about 40%, 80–82%, about 95%, plus 100%):

• **Part A: at translation time, no retraining.** Take a trained small gmm and, for a chosen share of words, send them to a random wrong slot (or to the correct slot, for 100%). Translate the dev set at each detection level and plot chrF against detection. This directly tests whether the translator benefits from correct detection.
• **Part B: during training.** Train gmm with a controlled share of root labels replaced by random roots (0%, 20%, 60%, 100% noise), which yields models with high to zero detection. This tests whether a model *trained* with better detection translates better. Two seeds per level; about 75 GPU-minutes per run.
• Repeat Part A on NLLB to see whether forcing correct detection helps a model that currently ignores the memory.

9.2 isiXhosa→English (language generality)

isiXhosa is agglutinative rather than root-and-pattern: words are built from a **stem** plus noun-class prefixes and verbal extensions. The same four families (A–D) will be run with slots tied to stems (instead of roots) and to affix classes (instead of patterns), using public parallel data and automatically derived segmentation labels. Success criterion: the same three questions (detection, use, translation) answered on a second, typologically different language.

9.3 Remaining items

• Two more seeds (44, 55) of the 1,024-slot small arm to test the +0.41 gain across seeds.
• Ablate the G1 fixes one at a time (gate loss, slot-content loss, label cleaning) to see which caused the stronger memory use.
• Exact-match accuracy on unseen words for the small G1 models; plain_bpe re-run on the current code.
• A single final evaluation on the AGE test set at the very end.

Full numbers: RESULTS_TAXONOMY.md and fix_results_all.csv (kaggle/analysis/). Each result comes from the run folders listed there.